pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

Branches and Pull Requests

WEEK 1 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: branches and pull requests

Today

1. Why branches: main is what is deployed
2. A branch of your own, read with diff_table
3. Two kinds of merge: fast-forward, and after a colleague
4. A conflict on purpose: keep both, the tests settle it
5. A pull request on GitHub, recorded and masked
6. Review by diff, before anything merges
7. Three ways to merge: the course squashes
8. Deleting merged branches, here and on the remote

Three functions, three branches

`diff_table`

what a change adds and removes

`branch_table`

ahead and behind main

`conflict_files`

what a merge left unmerged

No notebook cell reaches GitHub: the pull request is shown from its masked transcripts.

Why branches

main is what is deployed

From day 3 GitHub tests it on every change; from day 7 Vercel serves what it holds.

branch

work beside main

→

review

tested, read by another

→

merge

into main

→

delete

its job is done

A branch is a name pointing at a commit. It costs nothing.

A branch of your own

```
git switch -c diff-table
# edit, test
git commit -m "..."
git push -u origin diff-table
```

```
branch 'diff-table' set up to track 'origin/diff-table'.
To remote.git
 * [new branch]      diff-table -> diff-table
```

`diff_table(work, "main", "diff-table")`

```
            file  added  removed
     deploykit.py     18        0
test_deploykit.py     26        0
lines added: 44; removed: 0
```

A function and its test, nothing else touched: what a reviewer reads first.

Two kinds of merge

Fast-forward

main has not moved: Git slides it forward. No new commit.

```
exit code: 1
To remote.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'remote.git'
```

`fetch`	[ahead 1, behind 1]
`pull --ff-only`	Not possible to fast-forward, aborting.
`merge origin/main`	Merge made by the 'ort' strategy.
`push`	accepted this time

```
*   Merge remote-tracking branch 'origin/main'
|\
| * Ignore server logs
* | Add branch_table, with its test
|/
* Add diff_table, with its test
```

A conflict on purpose

```
deploykit.py
   174  <<<<<<< HEAD
   175  def conflict_files(folder) -> list[str]:
   183  =======
   184  def branch_table(folder):
   199  >>>>>>> main
```

Read both sides

HEAD

this branch: conflict_files

main

main's side: branch_table

both wanted

keep both, main's first

2

files in conflict

1

region in each

9

tests passed after

Git believes a file is resolved only when you stage it with git add.

A pull request is a branch plus a conversation

What the platform printed, masked

```
Recorded 2026-10-12, the course's account, masked
> git push -u origin units-in-readme
branch 'units-in-readme' set up to track 'origin/units-in-readme'.
remote: Create a pull request for 'units-in-readme' on GitHub by visi
remote:      https://github.com/student/pfi-deploy-demo/pull/new/unit
To https://github.com/student/pfi-deploy-demo.git
 * [new branch]      units-in-readme -> units-in-readme
```

gh pr view, masked

state	**OPEN**
author	**student (student)**
additions	**1,473**
deletions	**1,465**

A README change of a few lines, with over a thousand additions: section 2.6 reads why.

Review by diff

```
.gitignore  8
README.md  8
bondmath.py  1144
bondmath_excel_reference.csv  614
bondmath_excel_reference_tvm.csv  308
test_bondmath.py  856
```

The course's own pull request

README: the convention change. Five files: only their line endings, from core.autocrlf switched off inside a clone. Read the file list before you merge.

```
def future_value(amount: float, [-rate_pct:-]{+rate:+} float, years: float, frequency: int = [-2)-]{
Units: amount in dollars (or per 100 of face), [-rate_pct-]{+rate+} in percent, years in years.
Excel: [-=FV(rate_pct/100/frequency,-]{+=FV(rate/100/frequency,+} frequency*years, 0, -amount)
```

```
branch: future_value(100, 5.0, 10) = 162.8895
main:   future_value(100, 5.0, 10) = 163.8616
```

Three ways to merge

Merge commit

every commit, plus a merge commit

`merge --no-ff`

Squash

the branch's commits as one commit

`merge --squash`

Rebase

each commit copied, no merge commit

`commits replayed onto main`

```
Squash commit -- not updating HEAD
M  deploykit.py
M  test_deploykit.py
```

The course squashes your own pull requests: one change, one commit on main.

Deleting merged branches

```
ai-tidy         ahead  1  behind  1
branch-table    ahead  0  behind  3
conflict-files  ahead  2  behind  1
diff-table      ahead  0  behind  4
main            ahead  0  behind  0
```

1. `git push origin --delete <branch>`
2. `git branch -d <merged branch>`
3. `git branch -D`, only when you know why `-d` refused

```
error: the branch 'conflict-files' is not fully merged
```

A squash makes a new commit: the branch's own commits never reach main, so -d refuses.

Excel and Git

Excel side by side: none today

Excel has no branch, no merge, and no conflict to resolve: two copies stay two workbooks.

```
To remote.git
 - [deleted]         conflict-files
 - [deleted]         diff-table
```

The day's module commit: a squash merge of conflict-files. 9 tests on main.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: git checkout --theirs on every conflicted file: the reviewed side is dropped, and the merge still commits.

Code that runs is not the same as code that is right.

What you can do now

- main is what is deployed; work on a branch
- Read a change with diff_table before it merges
- Fast-forward, merge commit, a rejected push
- Resolve a conflict by reading both sides
- Pull requests, squash, and deleting branches

Week 1, Day 3 CI with GitHub Actions